UNLOX ACADEMY

VLSI Design Program, September 2026 Batch

MINOR PROJECT REPORT

PROJECT 04

VERILOG FUNDAMENTALS

(Data Types, Lexical Tokens and Programming Assignments)

Explained and modelled in Verilog HDL

Student Name	Thipperudra K T
Roll No.	4SM23EE037
Date	31 October 2026
Instructor	Kuldeep Sompura

VERILOG MINOR PROJECT

Part 1: Verilog Data Types

1.1 What are data types in Verilog?

A data type tells the simulator and the synthesis tool what kind of object a signal is, how many bits it has, and how it may be assigned. Verilog data types fall into two families. **Nets** model physical connections between hardware elements and do not store a value of their own. **Variables** model data storage and hold their value until the next assignment.

Table 1: Families of Verilog data types

Family	Types	Short description
Net	wire, tri, wand, wor, supply0, supply1	Connections driven by gates, modules or assign statements
Variable	reg, integer, real, time, realtime	Storage objects assigned in initial and always blocks

1.2 What is the difference between wire and reg?

Table 2: Comparison of wire and reg

Aspect	wire	reg
Family	Net	Variable
Meaning	A physical connection between elements	A storage object that keeps the last assigned value
Assigned by	assign statement, gate or module output	Statements inside initial or always blocks
Stores a value?	No, it follows its driver	Yes, until it is assigned again
Default value	z (high impedance)	x (unknown)
Port declaration	output wire (default)	output reg when assigned in an always block
Typical use	Connecting modules and combinational outputs	State registers, counters and testbench stimulus

```
wire y;
reg  q;
assign y = a & b;       // wire: driven continuously
always @(posedge clk)
    q <= d;             // reg: assigned in a procedural block
```

Fig. 1: wire driven by assign and reg driven by an always block

Note that the word reg does not always mean a flip-flop. A reg assigned in a combinational block such as `always @(*)` is synthesised as ordinary logic; it is called a reg only because it is assigned procedurally.

1.3 What is the difference between net and variable data types?

Table 3: Net data types compared with variable data types

Aspect	Net types	Variable types
Examples	wire, tri, wand, wor, supply0, supply1	reg, integer, real, time, realtime
Represents	Physical connections	Data storage
Needs a driver	Yes; undriven net shows z	No; holds value after assignment
Assigned with	Continuous assignment, gate or module output	Procedural assignment in initial or always
Multiple drivers	Allowed; the value is resolved (wand, wor, tri)	Not meaningful; last assignment wins
Initial value	z	x (0.0 for real)

1.4 What is the default value of a reg variable?

The default (initial) value of a reg is **x**, the unknown value, in every bit. The simulator cannot know what a storage element contains before it is assigned, so it uses x. A reg should therefore always be given a value, either by an initial block in a testbench or by a reset in a design.

Table 4: Default values of Verilog data types

Data type	Default value
reg (every bit)	x
integer	x
time	x
real	0.0
wire (undriven)	z

```
reg [3:0] r;
initial $display("r = %b", r);     // prints r = xxxx
```

Fig. 2: Default value of a reg

1.5 Can we assign a value to a wire inside an initial block?

No. An initial block is a procedural block, and the target of a procedural assignment must be a variable (reg, integer, time or real). A wire is a net, so a procedural assignment to it is an error reported at compile time. A net can only be given a value by a continuous assignment, a net declaration assignment, or by the output of a gate or module.

```
wire w;
initial w = 1'b1;       // ERROR: procedural assignment to a net

// Correct alternatives
reg  r;
initial r = 1'b1;       // 1. use a reg inside the initial block
wire w2;
assign w2 = 1'b1;       // 2. use a continuous assignment
wire w3 = 1'b1;         // 3. assign at declaration
```

Fig. 3: Illegal and legal ways to give a value

In a testbench the keywords `force` and `release` can temporarily override a net, but this is a special debugging feature, not an ordinary assignment, and it is not synthesisable.

1.6 What is the difference between integer and reg?

Table 5: Comparison of integer and reg

Aspect	integer	reg
Width	Fixed, at least 32 bits	User defined; 1 bit unless a range is given
Sign	Signed (two's complement)	Unsigned by default
Range (32 bits)	-2,147,483,648 to 2,147,483,647	0 to 2^n - 1 for n bits
Default value	x	x
Typical use	Loop counters, indexes, arithmetic in testbenches	Registers, buses, state variables, outputs

1.7 What is the purpose of the time data type?

The **time** data type is a 64-bit unsigned variable used to store simulation time. Its main use is to hold the value returned by the system function `$time` so that a testbench can record when an event occurred or measure the interval between two events. It exists only for simulation and is not synthesisable. The related type `realtime` stores time as a real number.

```
time t_start, t_end;
initial begin
    t_start = $time;
    #25;
    t_end = $time;
    $display("Elapsed = %0d", t_end - t_start);   // Elapsed = 25
end
```

Fig. 4: Measuring an interval with the time data type

1.8 What is the difference between real and integer?

Table 6: Comparison of real and integer

Aspect	integer	real
Value stored	Whole numbers only	Numbers with a fractional part
Representation	32-bit signed two's complement	64-bit floating point (IEEE 754 double)
Example values	-25, 0, 42	3.14, -0.5, 2.5e3
x and z values	Can hold x and z	Cannot; always a valid number
Default value	x	0.0
Synthesisable	Yes (as 32-bit register)	No; simulation only
Assignment of fraction	Rounded to a whole number	Stored with its fraction

Part 2: Lexical Tokens

2.1 What are lexical tokens in Verilog?

A lexical token is the smallest meaningful unit of Verilog source text. When a simulator or synthesis tool reads a file, its first step, called lexical analysis, splits the text into tokens in the same way that a reader splits a sentence into words. White space and comments only separate tokens and are then discarded; the remaining tokens are passed on to the parser, which checks that they form valid Verilog statements.

2.2 List the different types of lexical tokens

Table 7: Types of lexical tokens in Verilog

Token type	Description	Example
White space	Spaces, tabs and new lines that separate tokens	space, tab, newline
Comments	Explanatory text ignored by the tools	// note /* note */
Operators	Symbols that act on one, two or three operands	+ & == ?:
Numbers	Sized or unsized constants in binary, octal, decimal or hex	8'hA5 4'b1010 25
Strings	Text in double quotes, used in system tasks	"a = %d"
Identifiers	Names given by the designer to modules and signals	clk, data_out
Keywords	Reserved words with a fixed meaning	module, wire, always

2.3 What are keywords? Give five examples

Keywords are reserved words that have a predefined meaning in the language. They are written in lower case, cannot be used as names of signals or modules, and define the structure of the code.

Table 8: Five examples of Verilog keywords

Keyword	Use
`module / endmodule`	Start and end of a module definition
`input / output`	Declare the direction of a port
`wire`	Declares a net
`reg`	Declares a variable that stores a value
`always`	Starts a procedural block that repeats when triggered

Other keywords include `assign`, `initial`, `begin`, `end`, `integer`, `case` and `if`.

2.4 What are identifiers? Give three examples

An identifier is a name chosen by the designer for a module, port, wire, reg, parameter or instance. An identifier must begin with a letter or an underscore and may then contain letters, digits, the dollar sign and underscores. Identifiers are case sensitive, so `Count` and `count` are different names.

Table 9: Examples of identifiers

Identifier	Valid?	Reason
`clk`	Yes	Starts with a letter
`data_out`	Yes	Letters and underscore
`count_reg1`	Yes	Letters, underscore and digit
`2count`	No	Starts with a digit
`my-signal`	No	Hyphen is not allowed
`wire`	No	Reserved keyword

2.5 What are operators? Give five examples

Operators are symbols that perform an operation on one or more operands and produce a result. Verilog provides arithmetic, relational, equality, logical, bitwise, reduction, shift, conditional and concatenation operators.

Table 10: Five examples of Verilog operators

Operator	Name	Example	Result
`+`	Arithmetic addition	`5 + 3`	8
`&`	Bitwise AND	`4'b1100 & 4'b1010`	4'b1000
`==`	Equality	`6 == 6`	1 (true)
`&&`	Logical AND	`1 && 0`	0 (false)
`<<`	Shift left	`4'b0011 << 2`	4'b1100

2.6 What are comments in Verilog?

Comments are notes written by the designer to explain the code. The tools ignore them completely. Verilog supports two forms.

- **Single-line comment:** begins with `//` and runs to the end of the line.
- **Multi-line comment:** begins with `/*` and ends with `*/`, and may span many lines. Multi-line comments cannot be nested.

```
// This is a single-line comment
wire y;                 // comment after a statement

/* This is a multi-line comment.
   It can continue over several lines. */
reg q;
```

Fig. 5: Single-line and multi-line comments

2.7 Identify the operators in the following statement

The statement in the assignment sheet is cut off after the words "in the following state". Two typical statements are therefore analysed here to show the method; the same steps can be applied to any statement.

```
assign y = (a & b) | ~c;
```

Fig. 6: Statement 1

Table 11: Tokens of Statement 1

Token type	Tokens
Keyword	`assign`
Identifiers	`y, a, b, c`
Operators	& bitwise AND, | bitwise OR, ~ bitwise NOT
Assignment symbol	= (continuous assignment)
Separators	`( )` and `;`

```
if (count == 4'd9 && en) count <= count + 1;
```

Fig. 7: Statement 2

Table 12: Tokens of Statement 2

Token type	Tokens
Keyword	`if`
Identifiers	`count, en`
Numbers	`4'd9` and `1`
Operators	== equality, && logical AND, + addition
Assignment symbol	<= non-blocking assignment (not "less than or equal" here)

Part 3: Programming Assignments

Each program below is a complete Verilog module that can be compiled and run on any simulator. The expected outputs were traced from the code; $display prints at the moment the statement executes and %0d prints a number without leading spaces.

3.1 Program 1

Aim: Declare two reg variables, assign values using an initial block and display them using $display.

```
`timescale 1ns / 1ps
module program1;
    reg [7:0] a, b;              // two 8-bit reg variables
    initial begin
        a = 8'd25;               // decimal 25
        b = 8'hA5;               // hexadecimal A5 = decimal 165
        $display("a = %0d (binary %b, hex %h)", a, a, a);
        $display("b = %0d (binary %b, hex %h)", b, b, b);
    end
endmodule
```

Fig. 8: Source code of Program 1 (two reg variables)

Explanation. The module declares two 8-bit reg variables. The initial block executes once at time 0. Variable a receives the decimal constant 25 and b the hexadecimal constant A5. Because both are reg variables, procedural assignment inside the initial block is legal. $display then prints each value in decimal, binary and hexadecimal.

Expected output. Console output of Program 1.

```
a = 25 (binary 00011001, hex 19)
b = 165 (binary 10100101, hex a5)
```

3.2 Program 2

Aim: Declare an integer variable and perform addition of two numbers. Display the result.

```
module program2;
    integer num1, num2, sum;     // 32-bit signed variables
    initial begin
        num1 = 15;
        num2 = 27;
        sum  = num1 + num2;      // arithmetic addition
        $display("num1 = %0d, num2 = %0d", num1, num2);
        $display("Sum  = %0d + %0d = %0d", num1, num2, sum);
    end
endmodule
```

Fig. 9: Source code of Program 2 (integer addition)

Explanation. Three integer variables are declared. The numbers 15 and 27 are stored in num1 and num2, and their sum, calculated with the arithmetic operator +, is stored in sum. An integer is a 32-bit signed variable, so it can also hold negative results, for example when subtracting.

Expected output. Console output of Program 2.

```
num1 = 15, num2 = 27
Sum  = 15 + 27 = 42
```

3.3 Program 3

Aim: Declare a wire and implement an AND operation using the assign statement.

```
`timescale 1ns / 1ps
module program3;
    reg  a, b;                  // inputs driven in the initial block
    wire y;                     // net driven by assign
    assign y = a & b;           // continuous assignment: AND gate
    initial begin
        $monitor("time=%0d a=%b b=%b y=%b", $time, a, b, y);
        a = 0; b = 0;
        #10 a = 0; b = 1;
        #10 a = 1; b = 0;
        #10 a = 1; b = 1;
        #10 $finish;
    end
endmodule
```

Fig. 10: Source code of Program 3 (AND using assign)

Explanation. The wire y is driven by the continuous assignment `assign y = a & b;`, so it is re-evaluated whenever a or b changes, like the output of a real AND gate. The inputs a and b are reg variables because they are assigned in the initial block. The system task $monitor prints a new line each time a, b or y changes, giving the complete truth table of the AND gate.

Expected output. Console output of Program 3. The output y is 1 only when both inputs are 1.

```
time=0 a=0 b=0 y=0
time=10 a=0 b=1 y=0
time=20 a=1 b=0 y=0
time=30 a=1 b=1 y=1
```

3.4 Program 4

Aim: Write a Verilog program using reg, integer and time data types. Display their values.

```
`timescale 1ns / 1ps
module program4;
    reg [3:0] r;                // 4-bit reg
    integer   i;                // 32-bit signed integer
    time      t;                // 64-bit simulation time
    initial begin
        $display("Before assignment: r = %b, i = %0d", r, i);
        r = 4'b1010;
        i = -25;
        #50;                    // wait for 50 ns
        t = $time;              // capture current time
        $display("r = %b (binary) = %0d (decimal)", r, r);
        $display("i = %0d", i);
        $display("t = %0d ns", t);
    end
endmodule
```

Fig. 11: Source code of Program 4 (reg, integer and time)

Explanation. The first $display runs before any assignment and shows the default values: every bit of the reg is x and the integer is x. The reg is then loaded with 1010 (decimal 10) and the integer with -25. After a delay of 50 ns the time variable t captures the value of $time, which is 50 because the time unit is

1 ns.

Expected output. Console output of Program 4.

```
Before assignment: r = xxxx, i = x
r = 1010 (binary) = 10 (decimal)
i = -25
t = 50 ns
```

3.5 Program 5

Aim: Write a Verilog module for an OR gate using a continuous assignment.

```
`timescale 1ns / 1ps
module or_gate(input a, input b, output y);
    assign y = a | b;             // continuous assignment: OR gate
endmodule

module tb_or_gate;               // testbench
    reg  a, b;
    wire y;
    or_gate uut (.a(a), .b(b), .y(y));
    initial begin
        $monitor("time=%0d a=%b b=%b y=%b", $time, a, b, y);
        a = 0; b = 0;
        #10 a = 0; b = 1;
        #10 a = 1; b = 0;
        #10 a = 1; b = 1;
        #10 $finish;
    end
endmodule
```

Fig. 12: Source code of Program 5 (OR gate)

Explanation. The module or_gate has two input ports and one output port. The continuous assignment `assign y = a | b;` makes y follow the bitwise OR of a and b at all times. The testbench instantiates the module as uut, applies all four input combinations at 10 ns intervals and prints each result with $monitor.

Expected output. Console output of Program 5. The output y is 0 only when both inputs are 0.

```
time=0 a=0 b=0 y=0
time=10 a=0 b=1 y=1
time=20 a=1 b=0 y=1
time=30 a=1 b=1 y=1
```

3.6 Summary of Verification

Table 13: Verification of the five programs

Program	Concept demonstrated	Expected result
1	reg assigned in initial; $display with %d, %b, %h	a = 25, b = 165
2	integer arithmetic	15 + 27 = 42
3	wire driven by assign (AND)	y = 1 only for a = 1, b = 1
4	Default x; reg, integer, time	r = 1010, i = -25, t = 50
5	OR gate with continuous assignment	y = 0 only for a = 0, b = 0